

OJ 调试平台实验报告

1. 项目概述

本项目实现了一个面向课程实验与答辩演示的 OJ 调试平台，当前版本为 `v1.6`。系统采用 `FastAPI + Streamlit + SQLite` 技术栈，覆盖了题目管理、在线判题、提交日志、用户与权限、班级/作业/考试管理，以及 AI 智能命题、反 AI 作弊审查等功能。项目目标并不只是“能跑通”，而是尽量贴近真实 OJ 的使用体验：普通用户可以完成做题与查看个人数据，管理员可以维护题库、语言、用户、AI 配置，并对教学场景中的作业和考试进行管理。

从整体设计上看，系统分为五层：`Streamlit` 表现层、`common.py` 公共前端框架层、`FastAPI` 服务层、`userdb.py` 数据业务层，以及 `oj_judge/ai_engine` 核心能力层。这样的分层使页面渲染、接口鉴权、数据库操作、判题执行与 AI 任务彼此解耦，既便于定位问题，也方便在答辩中清楚说明每一类功能是如何落到代码实现上的。

2. 系统功能与设计

2.1 技术选型

本项目后端使用 `FastAPI`，前端使用 `Streamlit` 多页面模式，数据持久化采用 `SQLite`。之所以选择这组技术，是因为它们能够兼顾开发效率、可读性和演示友好性：

- `FastAPI` 路由清晰，接口文档天然可见，便于将系统功能拆分成标准 API。
- `Streamlit` 适合快速构建课程项目需要的多页面交互界面，能够直接展示表格、指标卡、按钮和输入组件。
- `SQLite` 足以支撑课程项目的数据规模，部署简单，适合本地运行和答辩演示。
- 判题核心单独放在 `oj_judge/` 目录，便于把运行、监控、判定逻辑从业务层剥离出来。
- AI 命题和配置逻辑独立到 `ai_engine.py` 与 AI 相关接口、页面中，避免污染普通 OJ 业务流。

2.2 模块划分

系统主要模块如下：

- 用户认证与权限模块：登录、注册、会话维持、角色区分、管理员入口控制。
- 题目模块：题目列表、题目详情、管理员新增/编辑/删除题目。
- 判题模块：代码提交、语言选择、判题结果展示、提交详情查看。

4. 提交日志模块：提交列表筛选、单条提交详情、测试点细节展示。
5. 教学场景模块：班级、作业、考试、面向普通学生的作业/考试做题入口。
6. AI 模块：AI 配置、AI 命题、题目草稿生成、造数脚本可视化编辑与重跑。
7. 反 AI 审查模块：管理员对提交进行多维风险分析，并在前端查看风险徽章和审查报告。
8. 前端公共框架模块：主题切换、中英双语、顶部导航、统一 API 调用与跳转。

2.3 系统架构

系统结构可以概括为如下五层：

1. 表现层：`webapp/app_streamlit.py` 与 `webapp/pages/*.py` 负责页面布局、交互控件、状态展示和路由跳转。
2. 公共框架层：`webapp/common.py` 负责统一 API 封装、当前用户信息读取、角色判断、主题和语言切换、导航栏渲染。
3. 服务层：`webapp/app_fastapi.py` 负责定义全部业务 API，包括认证、题目、提交、班级、作业、考试、AI 配置与 AI 命题。
4. 数据业务层：`webapp/userdb.py` 负责用户、会话、题目、提交、班级成员关系等数据的持久化与业务封装。
5. 核心能力层：`oj_judge/` 与 `webapp/ai_engine.py` 前者负责代码执行与判题，后者负责 AI 模型配置与生成流程。

这种结构的优点是：前端页面不直接访问数据库，而是统一调用后端 API；后端接口不直接内嵌复杂判题细节，而是将执行过程下沉到独立判题引擎；AI 功能作为增强能力存在，不会影响 OJ 主链路的稳定性。

3. 关键实现与难点

3.1 用户类型与权限区分

系统采用“认证、角色、接口鉴权、前端分流”四层权限设计。

第一层是身份认证。系统在用户登录成功后创建会话，并通过会话 ID 识别当前用户；`userdb.py` 中提供了 `create_session()` 和 `get_session_user()` 等函数，后端接口据此判断请求归属。密码不是明文保存，而是通过 `hash_password()` 进行哈希存储，默认管理员账号在系统初始化时自动写入，便于首次启动后直接管理平台。

第二层是角色区分。用户角色至少分为 `admin`、`user`、`banned` 三类；其中管理员拥有题目维护、用户管理、语言管理、AI 配置等额外能力，普通用户只能访问自己应有的数据，禁用用户则不能正常使用系统。

第三层是后端鉴权。 `app_fastapi.py` 中的 `require_login()` 装饰器统一控制接口访问，必要时通过 `allow_admin_only=True` 限制为管理员专属接口。这一层确保了即使前端被绕过，后端仍然能阻止越权请求。

第四层是前端分流。 `common.py` 中的 `current_user()` 和 `is_admin()` 在页面层决定是否显示“用户管理”“语言管理”“AI 配置”等管理员入口，实现“看得见什么”和“真正能调用什么”双重控制。

3.2 判题中的时间限制与空间限制

本项目并没有采用静态代码分析去猜测复杂度，而是采用“动态监控 + 大规模测试点”的方式去约束算法复杂度，这也更贴近真实 OJ 平台的做法。

时间限制方面， `oj_judge/executor.py` 中的 `CodeExecutor` 会启动子进程运行用户程序，并在 `_run_subprocess()` 中对执行线程做超时等待；若超过题目规定的时间限制，则终止对应进程，并将结果标记为 `timed_out`。后续 `oj_judge/judger.py` 再把这一执行结果映射为 `TLE`，展示到提交详情中。

空间限制方面，执行器在运行期间开启独立内存监控逻辑，在 `_start_memory_monitor()` 中使用 `psutil` 持续跟踪 RSS 峰值内存。一旦超过题目设置的内存上限，就立即标记 `memory_exceeded` 并终止进程。之后 `judger.py` 将其转换为 `MLE`。因此，系统不是通过“猜测这段代码是不是 $O(n^2)$ ”来限制复杂度，而是通过“真实执行 + 大数据点 + 资源阈值”来把不合格算法挡在评测过程中。

3.3 班级、作业、考试教学场景

为了满足课程项目从单纯刷题到教学管理的扩展需求，系统额外实现了班级、作业和考试模块。班级页面展示成员数量、关联作业和关联考试；作业和考试页面支持从教学任务上下文进入具体题目，跳转到判题页时还会携带 `assignment_id` 或 `exam_id`，让用户在“做题”时保留上下文。

这种设计的价值在于：同一套判题底层可以同时服务“题库刷题”和“教学任务提交”两种场景，而不需要复制两套系统。答辩时也可以清晰展示：系统不仅能做 OJ，还能做班级化作业与考试管理。

3.4 AI 命题功能的实现

AI 命题部分由 AI 配置页、AI 命题页、后端 AI 接口和 AI 引擎共同完成。管理员可以在 AI 配置页设置模型提供商、Base URL、模型名和密钥，然后在 AI 命题页输入需求发起命题。后端会以任务流方式推进，从“启动”“命题”“草稿”“样例生成”“校验”到“完成”逐步记录状态，前端再以“第 x/y 步 · 阶段名称”的形式展示实时进度。

本次迭代中特别修复了两个与 AI 命题直接相关的问题：

1. 修复了 `st.text_area()` 错误传入 `language='python'` 引发的页面崩溃。
2. 修复了原本难以理解的 `step` 文案，把它改成可读的阶段式进度提示。

同时，系统支持将 `generate_test.py` 造数脚本直接展示给管理员，并支持对脚本进行编辑与重跑，这一点对于演示“样例是如何真实生成的”非常关键。

3.5 反 AI 作弊审查的实现

除了 AI 命题，本项目还实现了面向管理员的反 AI 作弊审查能力。其核心实现位于

`webapp/ai_anti_cheat.py`，从五个维度对提交进行量化评分：`S1 AI生成概率`、`S2 代码相似度`、`S3 提交模式`、`S4 性能异常`、`S5 账号元数据`。系统按照加权公式计算综合分，并将风险等级划分为低、中、高三档。

在后端，`app_fastapi.py` 提供了两个接口：一个负责立即对指定提交运行审查并写入 `ai_reviews` 表，另一个负责读取最新审查结果。这样可以保证反 AI 审查不是写死在判题流程里，而是作为管理员可控的审核工具存在。

在前端，本次 `v1.6` 迭代把这一能力正式接入到了可视页面中：

1. 提交日志页对管理员显示 `AI 风险` 字段与风险文案，便于快速巡检近期提交。
2. 提交详情页新增“反AI审查”区域，管理员可手动触发审查，并直接查看综合分、五维得分和完整审查报告卡片。

这样的设计更符合教学场景：系统不会自动把某个提交直接判为作弊，而是为老师或管理员提供一套结构化参考，让人工审核更高效、更有依据。

3.6 本项目实现过程中的典型难点

在实现和验收过程中，我遇到的难点主要有以下几类：

1. 前后端状态同步问题。比如个人信息页的提交统计需要与最新提交实时同步，因此后端增加了 `recompute_user_stats()` 进行动态回算，避免出现“明明刚提交，个人信息还没更新”的问题。
2. 页面稳定性问题。此前某些页面在 `query` 参数缺失时会因为后续代码继续执行而白屏，后来通过在页面逻辑中增加 `if not problem: st.stop()` 之类的保护，解决了参数非法访问导致的崩溃。
3. AI 样例规模不合理问题。最初某些 AI 生成题目的样例过小，不足以体现集合、字典、前缀和等题型对复杂度的要求；后续通过改进样例脚本生成逻辑，让公开样例和隐藏点能更真实地区分不同算法复杂度。
4. 环境与进程稳定性问题。由于本项目用于本地演示，系统是否能在杀进程后正确重启也很重要，因此在此交付前专门执行了真实的进程清理与服务重启验证，确保 `5000` 和 `8501` 端口都能恢复服

务。

4. 成果展示与边界测试

本节所有截图均来自真实页面演示，截图文件统一存放在当前目录下的 `截图/` 子目录。为了满足“图文并茂”和“所有实现功能都要截图认证”的要求，下面按功能链路依次展示主要页面与关键能力。

4.1 登录注册与首页

图 4-1 展示了登录注册页面，证明系统具备用户登录与注册入口。

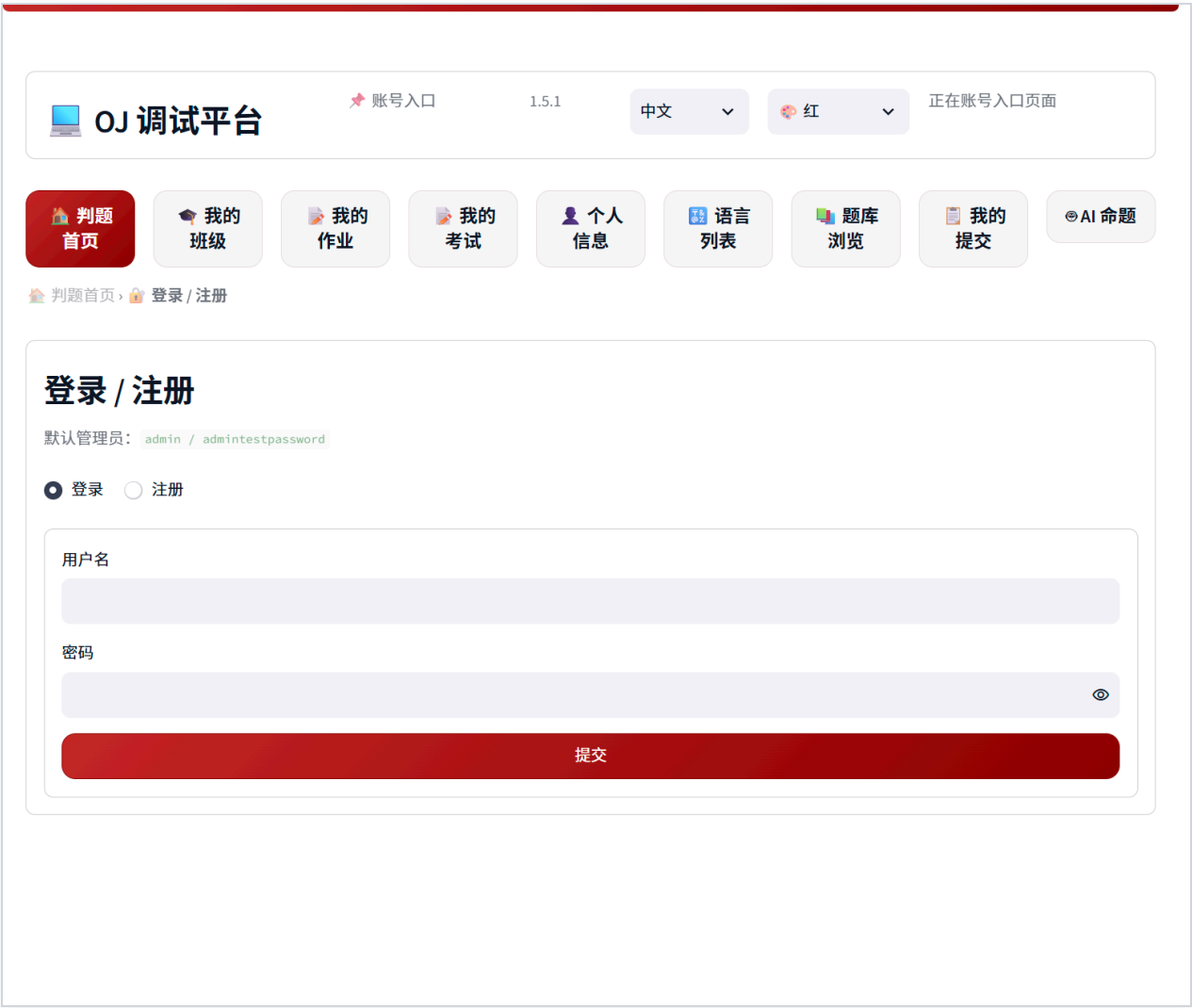


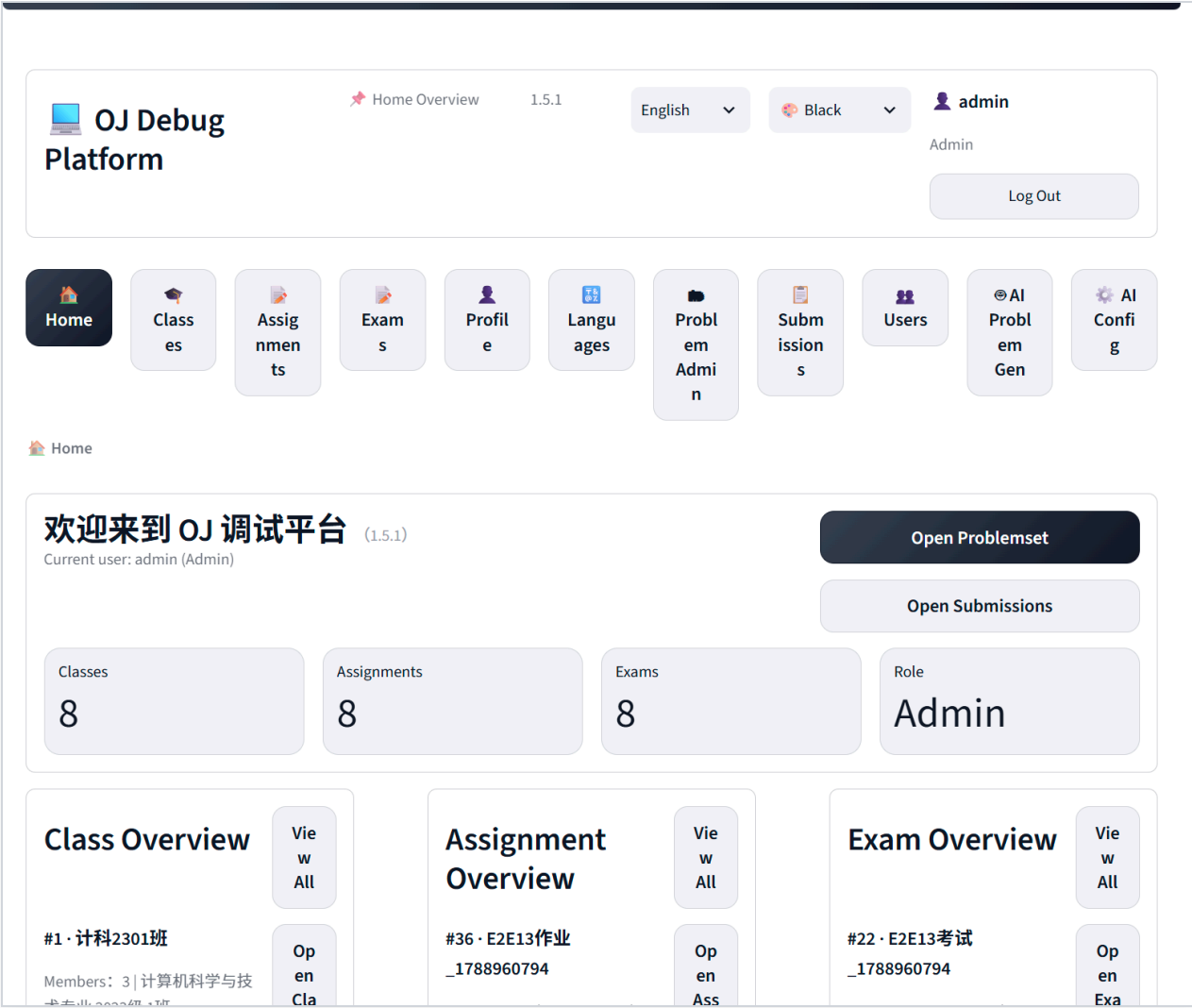
图 4-2 展示管理员首页，说明管理员登录后可以看到系统总览入口。



图 4-3 展示普通用户首页，说明不同角色的页面入口存在差异。



图 4-4 展示英文 + 黑色主题首页，说明系统支持国际化与主题切换。



4.2 用户中心、班级、作业、考试

图 4-5 为个人信息页面，展示了用户名、角色、注册时间、提交次数、通过题目数等数据。



图 4-6 为班级详情页面，可见成员数量、关联作业、关联考试等指标。



图 4-7 为作业列表/详情链路页面，证明学生可以从班级进入作业。

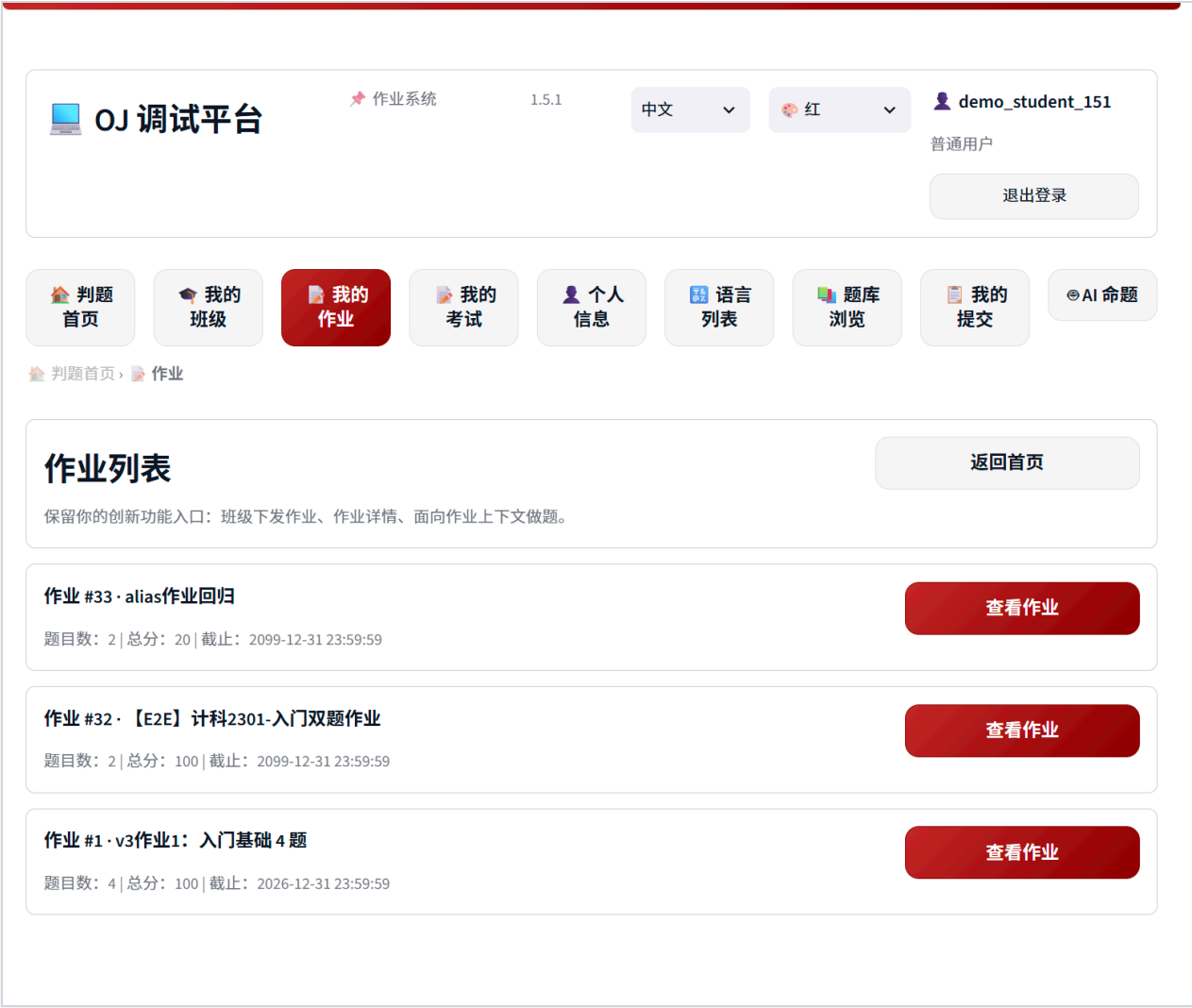


图 4-8 为单个作业详情页面，说明作业题目可以继续进入题目详情或做题页。

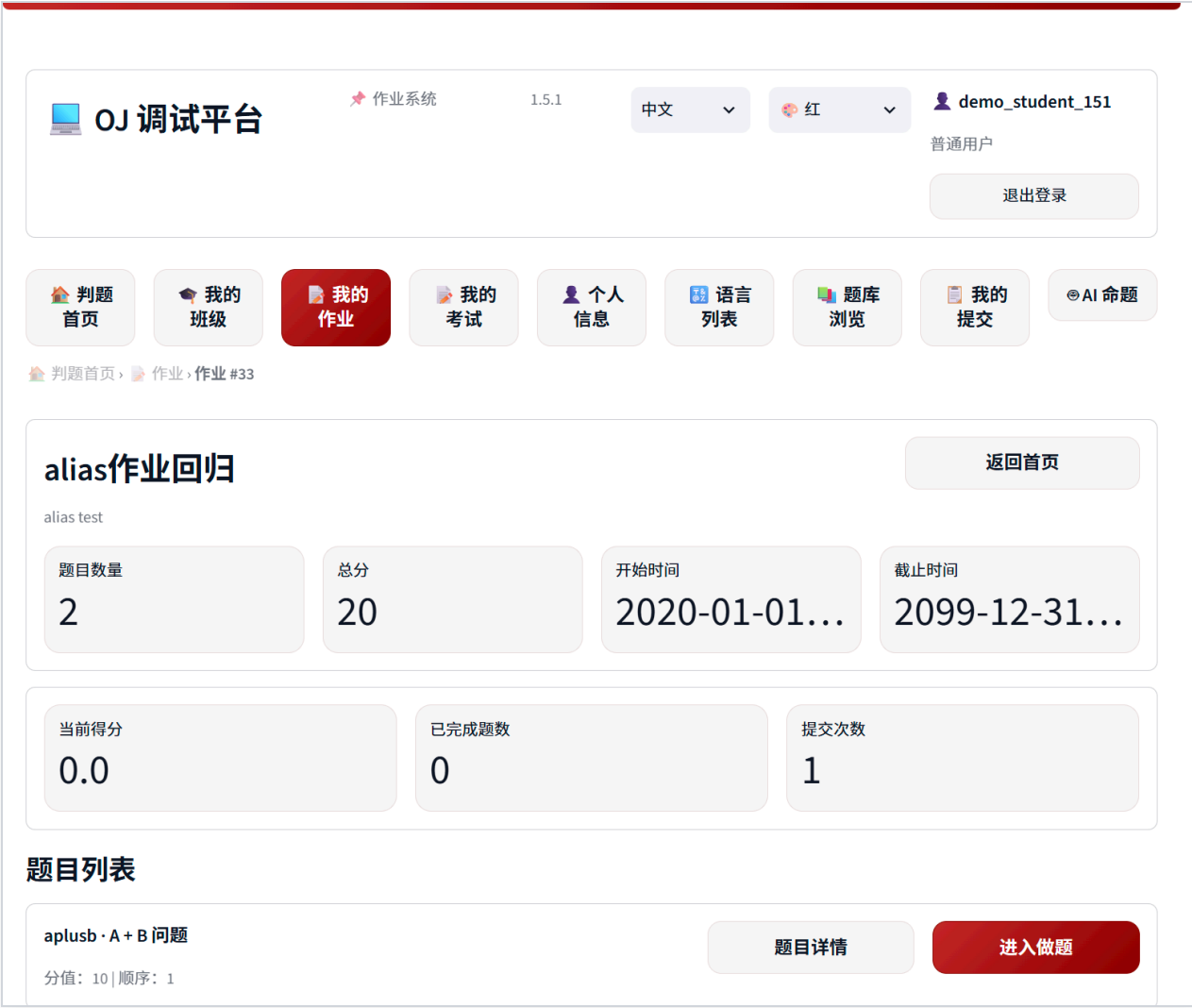
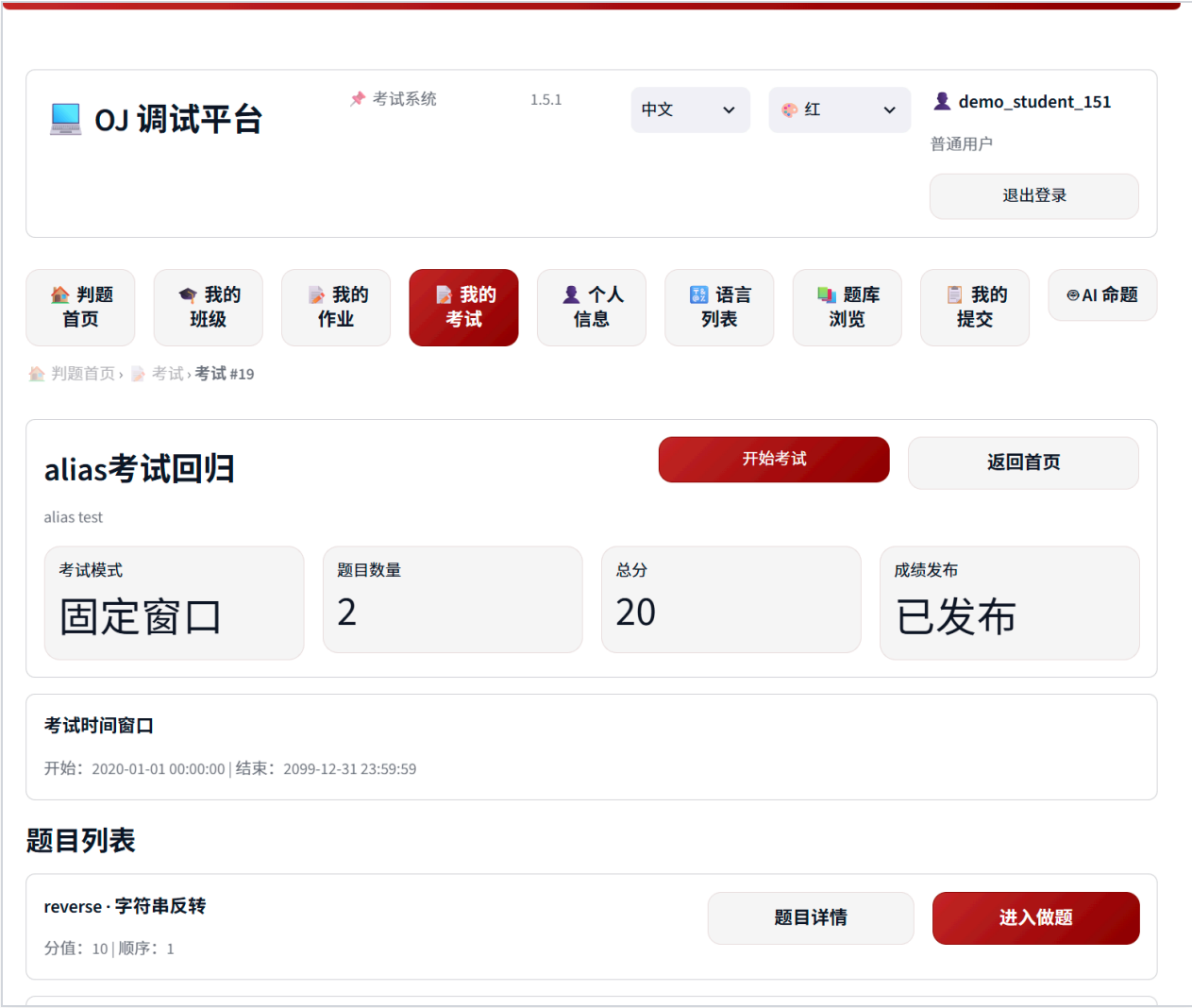


图 4-9 为单个考试详情页面，说明考试模式、题目数量以及进入做题入口已实现。



4.3 题库、题目详情与判题

图 4-10 为普通用户题库列表页，证明题库浏览、筛选和进入题目功能已实现。

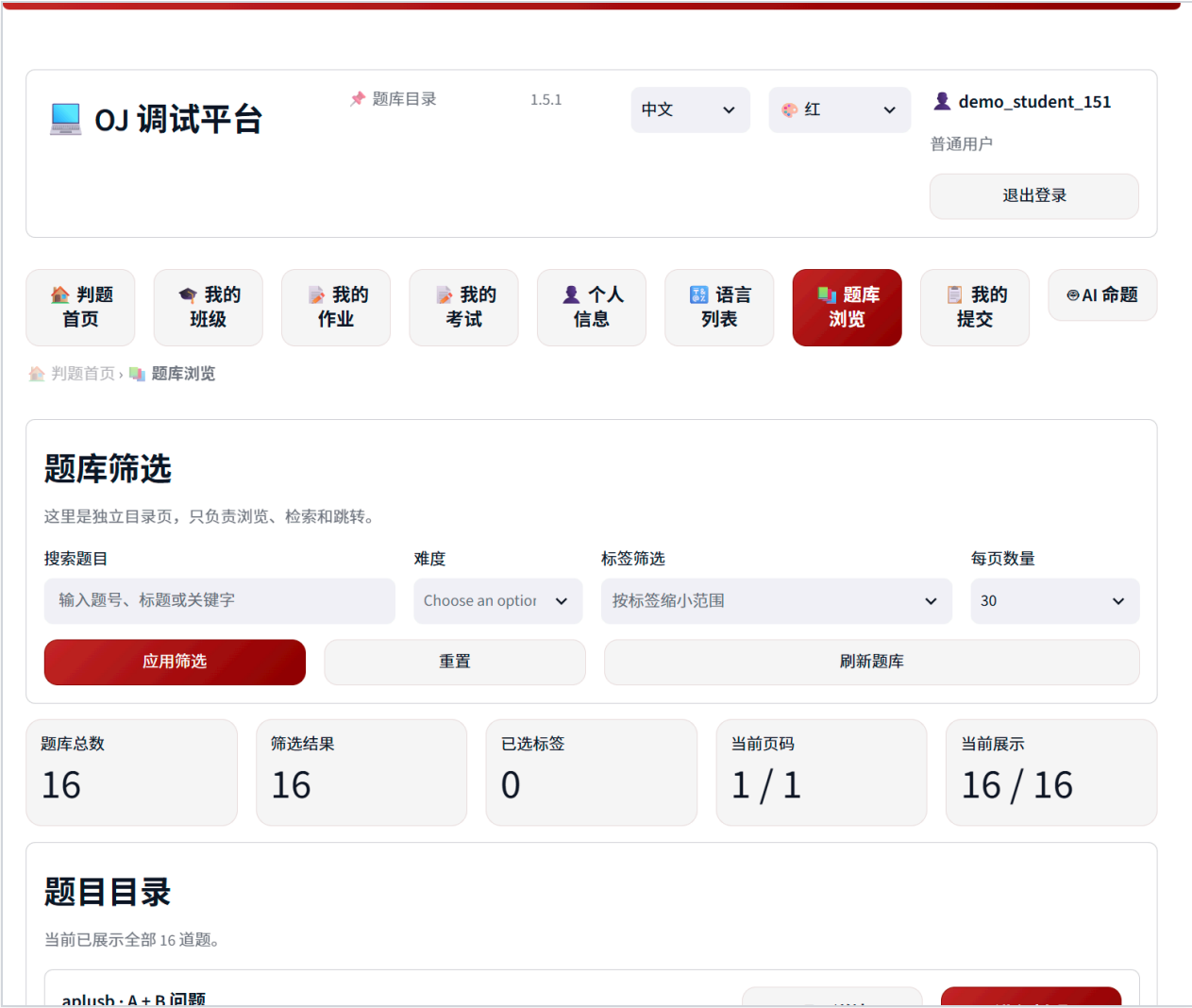


图 4-11 为题目详情页面，可见题面、题面样例、时间限制、内存限制等信息。



图 4-12 为判题页面，证明用户可以直接编写代码并提交评测。



4.4 提交日志与提交详情

图 4-13 为提交日志页面，说明系统支持查看用户提交记录。



图 4-14 为提交详情页面，展示测试点详情、编译与运行信息、基础信息等内容。



这一链路能够直接支撑边界测试展示，例如：语法错误时查看编译信息，超时程序查看 `TLE`，大内存程序查看 `MLE`，以及在不同用户身份下验证提交详情的可见性差异。

4.5 管理员后台能力

图 4-15 为用户管理页面，说明管理员可以查看并管理用户。



图 4-16 为语言管理页面，说明系统支持动态查看语言配置。



图 4-17 为管理员题目管理页面，说明管理员侧的题库维护入口已实现。



4.6 反 AI 审查展示

图 4-18 为管理员触发反 AI 审查后生成的审查卡片，展示了综合分、风险等级以及五个维度的评分细节。这一结果会在提交详情页的“反AI审查”区域中呈现，并同步影响提交日志页中的风险徽章。

AI 作弊风险审查报告 • Submission #291

用户: demo_student_151 | 题目: A + B 问题 | 综合分: 36 → ● 低风险

加权公式: $overall = 0.35 \cdot S1 + 0.25 \cdot S2 + 0.15 \cdot S3 + 0.15 \cdot S4 + 0.10 \cdot S5$; ≥ 70 ● 高风险, 40~69 ● 中风险, <40 ● 低风险。本审查仅为管理员提供参考, 不自动判定为作弊。

维度	名称	得分	细节 (本地规则计算值)
S1	AI生成概率	30	comment_rate=0.0; meaningless_var_rate=0.444; import_stdlib_ratio=0.0; char_entropy=3.934; avg_line_len=19.0; llm_score_used=None
S2	代码重复率	100	max_sim=1.0; high_match_count=3; count=6
S3	提交模式	0	recent_ac_rate=0.333; one_shot_ac=False
S4	性能异常	0	runtime_ratio_vs_peer_avg=0.609; memory_ratio_vs_peer_avg=0.975
S5	账号元数据	0	

4.7 AI 功能展示

图 4-19 为 AI 命题页面，展示了 AI 题目草稿、造数脚本和命题工作流入口。



图 4-20 为 AI 配置页面，展示了模型配置、DeepSeek 推荐参数与保存入口。



4.8 边界测试结果说明

本项目在展示系统效果之外，也重点验证了若干边界情况：

1. 权限边界：普通用户与管理员看到的页面入口不同，且后端接口有额外鉴权，避免仅靠前端隐藏按钮。
2. 页面参数边界：对缺失 `pid`、缺失提交 ID 等情况做了拦截，避免页面白屏。
3. 判题边界：对超时、超内存、运行错误、编译错误都能给出明确结果。
4. 数据同步边界：个人信息中的提交次数与通过题数能在新提交后重新计算。
5. 主题与语言边界：系统已验证中英双语和多主题切换可正常使用。
6. AI 任务边界：AI 命题进度不再只显示抽象 `step`，而是显示真实阶段名称。
7. 反 AI 审查边界：管理员可以对已有提交重复发起审查，审查结果会写入独立表并在提交列表中回显风险等级。

5. AI 使用说明

本项目在开发与完善过程中使用了 AI 工具辅助，但没有把 AI 当作“替代实现”的黑盒，而是把它放在“加速开发、辅助调试、辅助文档整理”的位置。整体工作流如下：

1. 先由人工明确需求、拆分功能点和验收标准。
2. 对已有代码进行静态阅读，确认实际架构和调用链。
3. 在局部功能上使用 AI 辅助生成或修改代码草案。
4. 所有关键改动都通过真实运行、页面点击、接口调用或截图来验证。
5. 对 AI 输出进行人工复核，确保与课程要求一致，避免生成“能看不能用”的内容。

在工具链上，项目内置的 AI 命题能力支持以 DeepSeek 为默认推荐配置；开发过程中也借助了 AI 辅助完成部分代码说明、问题定位和报告整理。若按工作量粗略估计，最终代码与文档仍以人工决策和人工验收为主，AI 更像“加速器”，而不是“直接交作业”的主体。

6. 代码规范与 Git 使用说明

课程评分标准明确要求正确使用 Git，并尽量按照 Conventional Commits 规范编写提交信息。本项目在这方面主要做了两件事。

第一，提交信息尽量遵循 Conventional Commits 风格。最近的提交记录中可以看到如下示例：

1. `release: v1.5.1 fix AI命题页报错与进度文案`
2. `fix(profile): 个人信息页 metric 清洗空值/换行 + 开始做题跳转题库 + 提交记录实时重算同步`
3. `fix(platform): complete v1.3 end-to-end workflow hardening`
4. `fix(ui): stabilize submission detail routing in v1.1.2`

这些提交基本都包含类型前缀与简洁说明，能够较好反映改动目的。

第二，避免将大文件和运行产物提交进 Git。项目根目录中的 `.gitignore` 明确忽略了 `data/`、`logs/`、`temp/`、`*.db`、`*.log`、各类临时调试脚本与缓存目录。实际检查中，本地虽存在体积较大的数据库文件，例如 `data/oj.db`，但未发现其被 Git 跟踪；这说明仓库层面已经在规避“大文件入库导致扣分”的风险。

7. 总结与建议

总体来看，本项目已经从一个基础 OJ 演化为一个更完整的课程实验平台。它既具备题目管理、判题、提交记录、权限控制这些核心 OJ 能力，也加入了班级/作业/考试与 AI 命题等更贴近教学应用的扩展功能。通过这次实现，我最大的收获有三点：

1. 课程项目不能只追求“功能表面完整”，还要重视稳定性、边界条件和可演示性。

2. 分层设计对后续调试和答辩说明帮助非常大，尤其是当系统功能越来越多时，清晰的模块边界能显著降低维护成本。
3. AI 工具确实能提高开发效率，但真正决定项目质量的，仍然是人工对需求、验证和细节的把控。

如果后续继续改进，我认为可以从以下几方面继续增强：

1. 将报告中的截图与自动化测试结果进一步联动，例如固定输出一组“演示数据快照”。
2. 为 AI 命题增加更多可解释性提示，例如明确展示测试点规模分层策略。
3. 将 PDF 导出流程自动化，减少最终交付时手工排版的工作量。

附录：功能实现 Index

以下索引写在正文后，用于如实指向各功能的实现位置，便于助教或答辩老师交叉检查。

功能	主要实现位置
版本号、主题、国际化	webapp/common.py:46-49
当前用户读取、管理员判断	webapp/common.py:140-148
页面 URL 跳转封装	webapp/common.py:182-209
顶部导航栏渲染	webapp/common.py:250-298
首页总览与入口按钮	webapp/app_streamlit.py:89-137
登录/注册页面	webapp/pages/🔒_登录注册.py:64-84
个人信息指标与入口按钮	webapp/pages/👤_个人信息.py:118-142
班级详情、成员/作业/考试 tabs	webapp/pages/🎓_班级.py:50-53
作业详情与进入做题	webapp/pages/📁_作业.py:31-37, 51-78
考试详情与开始考试/进入做题	webapp/pages/📝_考试.py:30-49, 57-82
题库筛选、题目详情入口、进入判题	webapp/pages/📖_题目管理.py:89, 179-192
题目详情、时间/内存限制、管理配置	webapp/pages/📄_题目详情.py:32-33, 56-61
判题页面、限制展示、提交后跳转详情	webapp/pages/⚖️_判题器.py:134-135, 258
提交日志入口、AI 风险徽章与详情跳转	webapp/pages/📋_提交日志.py:63, 129-149
提交详情、测试点/编译运行/基础信息/反AI审查 tabs	webapp/pages/📑_提交详情.py:43-118
用户管理页面	webapp/pages/👥_用户管理.py:60, 78, 89

功能	主要实现位置
语言管理页面	webapp/pages/  _语言管理.py:24, 61, 69-75, 108
AI 配置页面与保存配置	webapp/pages/  _AI配置.py:77-118
AI 命题进度显示、造数脚本编辑器	webapp/pages/  _AI命题.py:156-164, 253-261
反 AI 审查后端接口	webapp/app_fastapi.py:3807-3948
提交列表附加 AI 风险字段	webapp/app_fastapi.py:1145-1188, 3932-3948
反 AI 审查评分模型	webapp/ai_anti_cheat.py:31-433
登录鉴权装饰器	webapp/app_fastapi.py:250
当前登录用户接口	webapp/app_fastapi.py:355
注册接口	webapp/app_fastapi.py:383
登录接口	webapp/app_fastapi.py:420
语言列表接口	webapp/app_fastapi.py:555
题目列表接口	webapp/app_fastapi.py:647
题目详情接口	webapp/app_fastapi.py:707
创建提交接口	webapp/app_fastapi.py:851
提交详情接口	webapp/app_fastapi.py:1228
班级列表接口	webapp/app_fastapi.py:1726
班级关联作业接口	webapp/app_fastapi.py:1865
班级关联考试接口	webapp/app_fastapi.py:1886
AI 状态接口	webapp/app_fastapi.py:2364
AI 任务阶段推送	webapp/app_fastapi.py:2700-2781
AI 生成题目接口	webapp/app_fastapi.py:3316
AI 草稿脚本重跑接口	webapp/app_fastapi.py:3703
用户角色常量、默认管理员密码	webapp/userdb.py:60-66
密码哈希	webapp/userdb.py:92
用户统计重算	webapp/userdb.py:558
创建会话	webapp/userdb.py:579
根据会话读取用户	webapp/userdb.py:593

功能	主要实现位置
班级成员添加	webapp/userdb.py:2072
判题执行器入口	oj_judge/executor.py:54
子进程超时控制	oj_judge/executor.py:272-349
内存监控与 MLE 触发	oj_judge/executor.py:352-391
判题总控入口	oj_judge/judger.py:37-54
单测试点判定	oj_judge/judger.py:136-194
MLE/TLE 映射	oj_judge/judger.py:167-180
AI 模型默认配置	webapp/ai_engine.py:159-162
C++ 标程模板	webapp/ai_engine.py:134

附录：报告截图清单

截图文件	对应证明内容
截图/oj-auth-page.png	登录/注册
截图/oj-home-admin.png	管理员首页
截图/oj-home-user.png	普通用户首页
截图/oj-home-admin-black-en.png	英文界面 + 黑色主题
截图/oj-profile-user.png	个人信息与统计
截图/oj-class-detail-user.png	班级详情
截图/oj-assignment-detail-user.png	作业入口
截图/oj-assignment-single-user.png	作业详情
截图/oj-exam-single-user.png	考试详情
截图/oj-problems-user.png	普通用户题库
截图/oj-problem-detail-user.png	题目详情
截图/oj-judge-user.png	判题页面
截图/oj-submissions-user.png	提交日志
截图/oj-submission-detail-user.png	提交详情
截图/oj-users-admin.png	用户管理

截图文件	对应证明内容
截图/oj-languages-admin.png	语言管理
截图/oj-problems-admin.png	管理员题目管理
截图/oj-anti-ai-review-admin.png	反 AI 审查卡片
截图/oj-ai-admin.png	AI 命题
截图/oj-ai-config-admin.png	AI 配置

